
■ Centralisation des logs avec rsyslog

1. Contexte et objectifs du projet

1.1 Contexte

Dans une infrastructure informatique professionnelle, la **centralisation des journaux système** (logs = *journaux générés par les systèmes et applications*) est indispensable pour :

- la **supervision** des systèmes,
- le **diagnostic des incidents**,
- la **sécurité** (détection d'erreurs ou d'accès non autorisés),
- la **traçabilité** des actions.

Dans ce projet, une solution de centralisation des logs a été mise en place à l'aide de **rsyslog** (rsyslog = *service de journalisation avancé sous Linux*), intégrée à une infrastructure **Active Directory** existante.

1.2 Objectifs

Les objectifs du projet sont les suivants :

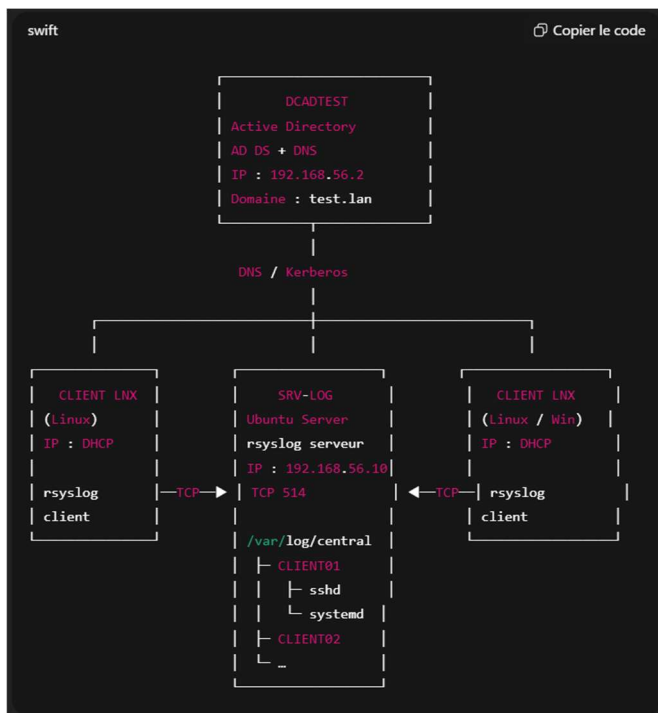
- Déployer un **serveur de centralisation des logs** sous Linux
 - Intégrer la solution dans un **environnement Active Directory**
 - Centraliser les logs de plusieurs machines clientes
 - Organiser automatiquement les journaux par :
 - machine source,
 - application,
 - date
 - Garantir une **résolution DNS fiable** et une **communication réseau stable**
-

2. Présentation de l'architecture

2.1 Vue globale de l'infrastructure

L'infrastructure repose sur les éléments suivants :

- **DCADTEST**
 - Contrôleur de domaine Active Directory
 - Service DNS intégré
 - Adresse IP statique
- **SRV-LOG**
 - Serveur Ubuntu Server
 - Service rsyslog
 - Intégré au domaine Active Directory
 - Adresse IP statique
- **Machines clientes**
 - Linux (et potentiellement Windows)
 - Membres du domaine
 - Envoi des logs vers SRV-LOG



2.2 Plan d'adressage IP

Machine	Rôle	Adresse IP	Type
DCADTEST	AD + DNS	192.168.56.2	Statique
SRV-LOG	rsyslog	192.168.56.10	Statique
Clients	Postes	DHCP	Dynamique

3. Mise en place du serveur SRV-LOG

3.1 Installation du système

- Système installé : **Ubuntu Server 24.04 LTS**
- Mode : serveur (sans interface graphique)
- Mise à jour du système via apt

```
root@srv-log:/home/luc# uname -a
Linux srv-log 6.8.0-71-generic #71-Ubuntu SMP PREEMPT_DYNAMIC Tue Jul 22 16:52:38 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
```

3.2 Configuration réseau statique

Le serveur SRV-LOG utilise une **adresse IP statique** afin d'assurer la cohérence avec le DNS Active Directory.

 **Fichier de configuration Netplan :**

network:

version: 2

renderer: networkd

ethernets:

ens33:

dhcp4: false

addresses:

- 192.168.56.130/24

nameservers:

addresses:

- 192.168.56.2

search:

- test.lan

```
root@srv-log:/home/luc# cat /etc/netplan/01-network.yaml
network:
  version: 2
  renderer: networkd
  ethernet:
    ens36:
      dhcp4: true
      dhcp4-overrides:
        use-dns: false
    ens37:
      dhcp4: false
      dhcp4-overrides:
        use-dns: false
      addresses:
        - 192.168.56.130/24
      nameservers:
        addresses:
          - 192.168.56.2
      search:
        - test.lan
```

4. Intégration à l'Active Directory

4.1 Objectif

L'intégration au domaine permet :

- l'utilisation du **DNS AD**,
- une meilleure lisibilité des machines,
- une architecture proche d'un environnement réel.

4.2 Jonction au domaine

Le serveur SRV-LOG a été joint au domaine Active Directory test.lan à l'aide de la commande :

sudo realm join test.lan -U Administrateur

```
root@srv-log:/home/luc# realm list
TEST.lan
  type: kerberos
  realm-name: TEST.LAN
  domain-name: test.lan
  configured: kerberos-member
  server-software: active-directory
  client-software: sssd
  required-package: sssd-tools
  required-package: sssd
  required-package: libnss-sss
  required-package: libpam-sss
  required-package: adcli
  required-package: samba-common-bin
  login-formats: %U@test.lan
  login-policy: allow-realm-logins
```

5. Déploiement de rsyslog

5.1 Présentation de rsyslog

rsyslog est un service de journalisation permettant :

- la collecte locale des logs,
- la réception de logs distants,
- le traitement et le stockage avancé des journaux.

5.2 Activation de la réception des logs distants

Le module TCP (Transmission Control Protocol = *protocole fiable*) est activé sur le serveur.

 **Extrait du fichier /etc/rsyslog.conf :**

```
module(load="imtcp")
```

```
input(type="imtcp" port="514")
```

```
# provides TCP syslog reception
module(load="imtcp")
input(type="imtcp" port="514")
```

5.3 Organisation des logs (template)

Un template personnalisé permet de stocker les logs automatiquement par machine, application et date.

 **Fichier /etc/rsyslog.d/10-central-logs.conf :**

```
template(
```

```
    name="CentralLogs"
```

```
    type="string"
```

```
    string="/var/log/central/%HOSTNAME%/%programname%/%$year%-%$month%-
    %$day%.log"
```

```
)
```

```
** ?CentralLogs
```

& stop

```
root@srv-log:/home/luc# tree -L 1 /var/log/central/ | less
/var/log/central/
├─ cl1
└─ srv-log
```

6. Configuration des machines clientes

6.1 Objectif

Les machines clientes doivent transmettre leurs journaux au serveur SRV-LOG via le réseau.

6.2 Configuration rsyslog côté client

 **Fichier /etc/rsyslog.d/90-forward-to-central.conf :**

**** @@srv-log.test.lan:514**

```
*_* @@srv-log.test.lan:514
```

6.3 Test de fonctionnement

Sur le client :

logger "TEST RSYSLOG CLIENT"

Sur le serveur :

ls /var/log/central

```
root@srv-log:/home/luc# ls /var/log/central/
cl1  srv-log
```

7. Résultats obtenus

- Centralisation effective des logs
- Classement automatique et structuré
- Communication fiable via TCP
- Intégration complète à l'Active Directory
- Architecture stable et évolutive

8. Mise en place du filtrage et de la catégorisation des journaux

8.1 Objectif de la catégorisation

Une fois la centralisation des journaux fonctionnelle, une étape supplémentaire a consisté à **améliorer la lisibilité et l'exploitation des logs** en mettant en place une **catégorisation automatique**.

L'objectif est de :

- distinguer les **journaux de sécurité**,
- séparer les **journaux applicatifs**,
- isoler les **journaux système**,
- tout en conservant une organisation **par machine source**.

Cette approche permet une **analyse plus rapide** et une **meilleure supervision** de l'infrastructure.

8.2 Principe de fonctionnement

La catégorisation repose sur l'analyse du champ programname (programname = *nom du programme générant le log*).

Selon l'application source, les journaux sont redirigés vers :

- un dossier **security** (authentification, privilèges),
- un dossier **web** (serveurs web),
- un dossier **system** (services système).

L'arborescence finale est structurée comme suit :

```
pgsql Copier le code
/var/log/central/<machine>/
├─ security/
│   └─ <application>/
│       └─ AAAA-MM-JJ.log
├─ web/
│   └─ <application>/
│       └─ AAAA-MM-JJ.log
└─ system/
    └─ <application>/
        └─ AAAA-MM-JJ.log
```

8.3 Configuration rsyslog côté serveur

Un fichier de configuration dédié a été créé afin de définir des **templates dynamiques** et des **règles conditionnelles**.

 **Fichier : /etc/rsyslog.d/15-categories.conf**

```
template(name="T_Security" type="string"
    string="/var/log/central/%hostname%/security/%programname%//%$year%-
    %$month%-/%$day%.log")
```

```
template(name="T_Web" type="string"
    string="/var/log/central/%hostname%/web/%programname%//%$year%-
    %$month%-/%$day%.log")
```

```
template(name="T_System" type="string"
    string="/var/log/central/%hostname%/system/%programname%//%$year%-
    %$month%-/%$day%.log")
```

```
if ($programname == "sshd" or $programname == "sudo") then {
    action(type="omfile" dynaFile="T_Security")
    stop
}
```

```
if ($programname == "apache2" or $programname == "nginx") then {
    action(type="omfile" dynaFile="T_Web")
    stop
}
```

```
if ($programname == "systemd" or $programname == "kernel") then {
    action(type="omfile" dynaFile="T_System")
```



```
stop  
}
```

```
root@srv-log:/home/luc# rsyslogd -N1  
rsyslogd: version 8.2312.0, config validation run (level 1), master config /etc/rsyslog.conf  
rsyslogd: End of config validation run. Bye.
```

8.4 Validation et redémarrage du service

Après modification de la configuration, une validation syntaxique est réalisée afin d'éviter toute erreur de démarrage :

```
rsyslogd -N1
```

Puis le service est redémarré :

```
systemctl restart rsyslog
```

```
root@srv-log:/home/luc# systemctl status rsyslog  
• rsyslog.service - System Logging Service  
   Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)  
   Active: active (running) since Sun 2025-12-14 12:35:47 CET; 20min ago  
   TriggeredBy: • syslog.socket  
   Docs: man:rsyslogd(8)  
         man:rsyslog.conf(5)  
         https://www.rsyslog.com/doc/  
   Process: 2862 ExecStartPre=/usr/lib/rsyslog/reload-apparmor-profile (code=exited, status=0/SUCCESS)  
   Main PID: 2867 (rsyslogd)  
     Tasks: 9 (limit: 19039)  
    Memory: 1.8M (peak: 5.3M)  
       CPU: 97ms  
   CGroup: /system.slice/rsyslog.service  
           └─2867 /usr/sbin/rsyslogd -n -iNONE  
  
déc. 14 12:35:47 srv-log systemd[1]: Starting rsyslog.service - System Logging Service...  
déc. 14 12:35:47 srv-log rsyslogd[2867]: imuxsock: Acquired UNIX socket '/run/systemd/journal/syslog' (fd 3) from systemd. [v8.2312.0]  
déc. 14 12:35:47 srv-log systemd[1]: Started rsyslog.service - System Logging Service.  
déc. 14 12:35:47 srv-log rsyslogd[2867]: rsyslogd's groupid changed to 104  
déc. 14 12:35:47 srv-log rsyslogd[2867]: rsyslogd's userid changed to 103  
déc. 14 12:35:47 srv-log rsyslogd[2867]: [origin software="rsyslogd" swVersion="8.2312.0" x-pid="2867" x-info="https://www.rsyslog.com"] start
```

9. Tests de la catégorisation

9.1 Tests côté client

Depuis une machine cliente, des événements sont générés afin de vérifier le routage correct des journaux :

```
sudo sudo ls
```

```
logger -t sshd "TEST SSHD CATEGORIE"
```

```
root@cl1:/home/luc# logger -t kernel "tes kernet"
```

9.2 Vérification côté serveur

Sur le serveur de centralisation :

```
tree -L 5 /var/log/central/<nom_machine>
```

Les journaux apparaissent correctement dans les dossiers correspondant à leur catégorie.

```
root@srv-log:/home/luc# tree -L 5 /var/log/central/  
/var/log/central/  
└─ cl1  
    ├── security  
    │   ├── sshd  
    │   │   └─ 2025-12-14.log  
    │   └─ sudo  
    │       └─ 2025-12-14.log  
    └─ system  
        ├── kernel  
        │   └─ 2025-12-14.log  
        └─ systemd  
            └─ 2025-12-14.log  
  
8 directories, 4 files
```

```
root@srv-log:/home/luc# cat /var/log/central/cl1/system/kernel/2025-12-14.log  
2025-12-14T11:45:16+01:00 cl1 kernel: tes kernet
```

10. Apports de la solution mise en place

La solution développée apporte plusieurs bénéfices :

- organisation claire et hiérarchisée des journaux,
- séparation logique des événements sensibles,
- amélioration de la supervision et du diagnostic,
- réduction du temps d'analyse en cas d'incident,
- architecture évolutive sans modification côté client.

11. Structuration des journaux au format JSON

11.1 Objectif de la structuration en JSON

Après la mise en place de la centralisation et de la catégorisation des journaux, une nouvelle étape a consisté à **structurer les logs de sécurité au format JSON** (JSON = *JavaScript Object Notation, format de données structuré clé/valeur*).

L'objectif est de :

- rendre les journaux **exploitables automatiquement**,
- faciliter leur **analyse par un programme**,
- préparer la génération de **statistiques et d'alertes**.

Cette structuration concerne prioritairement les **journaux de sécurité** (authentification, élévation de privilèges).

11.2 Principe de fonctionnement

Les journaux de sécurité sont désormais :

- stockés en **format texte** pour la consultation humaine,
- **dupliqués volontairement** en **format JSON** pour l'analyse automatisée.

Chaque événement est transformé en un objet JSON contenant des champs précis :

- date et heure,
- machine source,
- application,
- niveau de gravité,
- message.

Exemple de journal structuré :

```
{  
  "timestamp": "2025-12-14T10:47:18+01:00",  
  "hostname": "cl1",  
  "program": "sshd",  
  "severity": "warning",  
  "facility": "auth",  
  "message": "Failed password for root from 192.168.56.20"  
}
```

```
root@srv-log:/home/luc# cat /var/log/central/cl1/security-json/2025-12-14.json
{"timestamp":"2025-12-14T13:25:17+01:00","hostname":"cl1","program":"sshd","severity":"notice","facility":"user","message":" test json structure ok"}
{"timestamp":"2025-12-14T13:25:18+01:00","hostname":"cl1","program":"sudo","severity":"notice","facility":"authpriv","message":" root : TTY=pts/0 ; PWD=/home/luc ; USER=root ; COMMAND=/usr/bin/ls"}
{"timestamp":"2025-12-14T13:25:18+01:00","hostname":"cl1","program":"sudo","severity":"info","facility":"authpriv","message":" pam_unix(sudo:session): session opened for user root(uid=0) by luc(uid=0)"}
{"timestamp":"2025-12-14T13:25:18+01:00","hostname":"cl1","program":"sudo","severity":"info","facility":"authpriv","message":" pam_unix(sudo:session): session closed for user root"}
```

11.3 Mise en place des templates JSON

Deux templates rsyslog ont été définis :

- un template pour le **chemin dynamique du fichier JSON**,
- un template pour le **format des données JSON**.

 **Fichier : /etc/rsyslog.d/12-json-template.conf**

```
template(name="JsonSecurityFile" type="string")
```

```
    string="/var/log/central/%hostname%/security-json/%$year%-%$month%-%$day%.json")
```

```
template(name="JsonFormat" type="list") {
```

```
    constant(value="{")
```

```
    constant(value="\timestamp\":"\") property(name="timereported")
    dateFormat="rfc3339")
```

```
    constant(value="\","hostname\":"\") property(name="hostname")
```

```
    constant(value="\","program\":"\") property(name="programname")
```

```
    constant(value="\","severity\":"\") property(name="syslogseverity-text")
```

```
    constant(value="\","facility\":"\") property(name="syslogfacility-text")
```

```
    constant(value="\","message\":"\") property(name="msg")
```

```
    constant(value=""}\n")
```

```
}
```

```
root@srv-log:/home/luc# cat /etc/rsyslog.d/12-json-template.conf
template(name="JsonSecurityFile" type="string"
    string="/var/log/central/%hostname%/security-json/%$year%-%$month%-%$day%.json")

template(name="JsonFormat" type="list") {
    constant(value="{")
        constant(value="\t\t\"timestamp\":"")
            constant(value="\t\t\t\"hostname\":"")
                constant(value="\t\t\t\t\"program\":"")
                    constant(value="\t\t\t\t\t\"severity\":"")
                        constant(value="\t\t\t\t\t\t\"facility\":"")
                            constant(value="\t\t\t\t\t\t\t\"message\":"")
                                constant(value="\t\t\t\t\t\t\t\t"}\n")
        property(name="timereported" dateFormat="rfc3339")
        property(name="hostname")
        property(name="programname")
        property(name="syslogseverity-text")
        property(name="syslogfacility-text")
        property(name='msg')
    }
}
```

11.4 Intégration de la sortie JSON dans la règle de sécurité

Les journaux de sécurité sont traités dans une seule règle rsyslog, permettant :

- l'écriture des logs au format texte,
- l'écriture simultanée des logs au format JSON,
- l'arrêt du traitement afin d'éviter les doublons non souhaités.

 Extrait du fichier : /etc/rsyslog.d/15-categories.conf

```
if ($programname == "sshd" or $programname == "sudo") then {
    action(type="omfile" dynaFile="T_Security" createDirs="on")
    action(type="omfile" dynaFile="JsonSecurityFile" template="JsonFormat"
createDirs="on")
    stop
}
```

```
GNU nano 7.2 /etc/rsyslog.d/15-categories.conf
template(name="T_Security" type="string"
string="/var/log/central/%hostname%/security/%programname%/%year%-%month%-%day%.log")

template(name="T_Web" type="string"
string="/var/log/central/%hostname%/web/%programname%/%year%-%month%-%day%.log")

template(name="T_System" type="string"
string="/var/log/central/%hostname%/system/%programname%/%year%-%month%-%day%.log")

# catégorie : sécurité

if ($programname == "sshd" or $programname == "sudo") then {
    # log text ( catégorieiser)
    action(type="omfile" dynaFiles="T_Security" createdirs="on")
    # log json (structure)
    action(type="omfile" dynaFiles="JsonSecurityFile" template="JsonFormat" createdirs="on")
    stop
}

# catégorie : web

if ($programname == "apache2" or $programname == "nginx") then {
    action(type="omfile" dynaFiles="T_Web")
    stop
}

# catégorie : système

if ($programname == "systemd" or $programname == "kernel") then {
    action(type="omfile" dynaFiles="T_System")
    stop
}
```

11.5 Validation et redémarrage du service

Avant toute mise en production, la configuration rsyslog a été validée :

rsyslogd -N1

Puis le service a été redémarré :

systemctl restart rsyslog

```
root@srv-log:/home/luc# rsyslogd -N1
rsyslogd: version 8.2312.0, config validation run (level 1), master config /etc/rsyslog.conf
rsyslogd: End of config validation run. Bye.
```

12. Tests de la structuration JSON

12.1 Génération d'événements de sécurité

Depuis une machine cliente :

sudo sudo ls

logger -t sshd "TEST JSON SECURITE"

12.2 Vérification côté serveur

Sur le serveur de centralisation :

ls /var/log/central/<nom_machine>/security-json

cat /var/log/central/<nom_machine>/security-json/AAAA-MM-JJ.json

Les journaux apparaissent correctement sous forme d'objets JSON exploitables.

```
root@srv-log:/home/luc# cat /var/log/central/cl1/security-json/2025-12-14.json
{"timestamp":"2025-12-14T13:25:17+01:00","hostname":"cl1","program":"sshd","severity":"notice","facility":"user","message":" test json structure ok"}
{"timestamp":"2025-12-14T13:25:18+01:00","hostname":"cl1","program":"sudo","severity":"notice","facility":"authpriv","message":"    root : TTY=pts/0 ; PWD=/home/luc ; USER=root ; COMMAND=/usr/bin/ls"}
{"timestamp":"2025-12-14T13:25:18+01:00","hostname":"cl1","program":"sudo","severity":"info","facility":"authpriv","message":" pam_unix(sudo:session): session opened for user root(uid=0) by luc(uid=0)"}
{"timestamp":"2025-12-14T13:25:18+01:00","hostname":"cl1","program":"sudo","severity":"info","facility":"authpriv","message":" pam_unix(sudo:session): session closed for user root"}
{"timestamp":"2025-12-14T13:28:09+01:00","hostname":"cl1","program":"sshd","severity":"notice","facility":"user","message":" test json template order"}
{"timestamp":"2025-12-14T13:28:12+01:00","hostname":"cl1","program":"sudo","severity":"notice","facility":"authpriv","message":"    root : TTY=pts/0 ; PWD=/home/luc ; USER=root ; COMMAND=/usr/bin/ls"}
{"timestamp":"2025-12-14T13:28:12+01:00","hostname":"cl1","program":"sudo","severity":"info","facility":"authpriv","message":" pam_unix(sudo:session): session opened for user root(uid=0) by luc(uid=0)"}
{"timestamp":"2025-12-14T13:28:12+01:00","hostname":"cl1","program":"sudo","severity":"info","facility":"authpriv","message":" pam_unix(sudo:session): session closed for user root"}
```

13. Intérêt de la double représentation des logs

La conservation des journaux en **double format** répond à un objectif précis :

- **Format texte** : lecture rapide et diagnostic manuel,
- **Format JSON** : analyse automatisée et exploitation avancée.

Cette approche s'inspire des pratiques utilisées dans les environnements professionnels de supervision et de sécurité.